DEPARTAMENTO DE
INGENIERÍA
INFORMÁTICA
UNIVERSIDAD DE SANTIAGO DE CHILE

Análisis de Algoritmos y Estructuras de Datos

Unidad 1: Algoritmos en pseudocódigo y lenguaje C

LABORATORIO semana 3

Punteros y pasada por referencia, matrices, archivos y registros

Semestre 2-2026

Objetivos

- Aplicar variables de tipo puntero y pasada por referencia en funciones C que implementan algoritmos.
- Utilizar arreglos y matrices para procesar datos de entradas que se encuentran almacenados en archivos.
- Programar en C algoritmos que usen registros o struct C (estructuras que agrupan un conjunto de variables de distinto tipo) para implementar algoritmos

DEPARTAMENTO DE
INGENIERÍA
INFORMÁTICA
UNIVERSIDAD DE SANTIAGO DE CHILE

Estructura de un programa en C

Recordemos

```
#include<stdio.h>
#include<stdlib.h>
#define TAM_FIJO 4
```

Declaración de bibliotecas Preprocesador

```
int suma(int a){
    int acum, i;
    if (a<0){
        return -1;
    }
    for(i=0;i<a;i++){
        acum = acum + i; //acum += i;
    }
    return acum;
}
```

Declaración de **Funciones que implementan algoritmos**

retorno

```
void main(){
    valor = suma(20);
    printf("Valor de suma: %d\n", valor);
}
```

La función **main provee de las entradas** usadas en los algoritmos (funciones) y **muestra las salidas o sus resultados**

- **#include** inicializa el entorno de trabajo, hace referencia a librerías/bibliotecas.
- **stdio.h** es una librería C que permite ejecutar funciones de entradas y salidas de datos. stdlib.h y para manejo de memoria. Math. Para uso de funciones matemáticas, etc.
- **printf** y **scanf** son funciones de la librería stdio.h

  scanf(*"formato_entr1 formato_entr2..."*, &var1, &var2,...);

  printf("mensaje y *formato_sal1 formato_sal2*", var1, var2,...);
- // ó /* */ : comentarios
- Formatos de entrada/salida: %d, %f, %c, etc
- Algunos delimitadores:
  - Paréntesis **( )** : Indican los parámetros de las funciones.
  - Llaves **{ }** : Delimitan sentencias compuestas (bloques de código)
  - Punto y coma **;** : Indica el final de la sentencia.

DEPARTAMENTO DE
INGENIERÍA
INFORMÁTICA
UNIVERSIDAD DE SANTIAGO DE CHILE

Lectura de archivos
(guarda datos en **arreglo**)

Dado un archivo con n elementos (un elemento por fila) **donde n se encuentra en la primera fila**, guardar dichos elementos en un arreglo y retornarlo.

dataA.txt

```
10
7
3
6
4
21
9
55
45
100
20
```

Se asume que el archivo está dado. Se recibe el nombre del archivo para ser abierto dentro del algoritmo

```
NUM[] leer_datos_fil(CHAR nom_archivo, NUM *n)
  FILE fp = OPENFILE(nom_archivo, "r")
  READFILE(fp,1,n) //lee desde fp 1 valor y lo guarda en n
  NUM array[*n]
  FOR i=0 TO *n-1 DO
      READFILE(fp, 1, array[i])
  END-FOR
  CLOSEFILE(fp)
  RETURN  array
```

La entrada *n recibe la dirección de memoria para acceder al contenido por referencia (la actualización de su contenido podrá verlo reflejado otros algoritmos.

```
gcc leerFil.c  -o leerFil.exe
./leerFil.exe dataA.txt
```

```c
#include<stdio.h>
#include<stdlib.h>
#include<math.h>

int* leer_datos_Fil(char *nom_archivo, int *n) {
  FILE *fp = fopen(nom_archivo, "r");
  if (fp==NULL){
    printf("\n Archivo no existe\n");
    return NULL;
  }
  fscanf(fp, "%d", n);
  int *array = malloc(*n * sizeof(int));
  for (int i = 0; i < *n; i++)
    fscanf(fp, "%d", &array[i]);
  fclose(fp);
  return array;
}
int main(int argc,  char *argv[]){
    int n=0;
    if (argc != 2)
        return 1;
    int *A = leer_datos_Fil(argv[1], &n);
    if ( A == NULL )
        return 1;
    for (int i = 0; i < n; i++)
        printf("%d ", A[i]);
    free(A);
    return 0;
}
```

Lee datos del archivo desde la segunda fila.

***A** recibe la @ del primer elemento del array creado

paso por referencia el **n** para que pueda usarlo aquí en el main

Cada llamada a `malloc` debe corresponder a una llamada a `free`.

DEPARTAMENTO DE
INGENIERÍA
INFORMÁTICA
UNIVERSIDAD DE SANTIAGO DE CHILE

Matrices

Ejemplo usando pseudocódigo:

n=5 //filas
m=4 //columnas
NUM M[n][m]]

Cada elemento se accede usando una posición:

M[0][0], M[0][1], M[0][2], M[0][3]
M[1][0], M[1][1], M[1][2], M[1][3]
M[2][0], M[2][1], M[2][2], M[2][3]
…..

En pseudocódigo, una matriz se recorre:

```
FOR i = 0 TO n-1 DO
    FOR j = 0 TO m-1 DO
        instrucciones para procesar M[i][j]
    END-FOR
END-FOR
```

```c
#include <stdlib.h>
#include <stdio.h>
#define n_FIJO 10
#define m_FIJO 4

//(1): Arreglo de tamaño fijo ingresando datos por teclado
int M[n_FIJO][m_FIJO ];
int n=n_FIJO, m=m_FIJO

//(2): Inicializando un arreglo y dejamos que C determine el tamaño
int M[2][5] = { { 15, 17, 20, 25, 10 }, { 18, 20, 21, 23, 18 }};
int n = sizeof(M) / sizeof(M[0]);
int m = sizeof(M[0]) / sizeof(M[0][0]);

//(3): Si n,m los obtengo en tiempo de ejecución. Usar malloc y free
scanf("%d %d", &n, &m);
int **M = malloc(n * sizeof(int *)); //memoria para las n filas
for(int i=0;i<n;i++){
   M[i] = malloc(m * sizeof(int)); //para fila i reserva m columnas
}
/* En los tres casos la matriz se procesa de la misma forma */
for (int i = 0; i<n; i++) {
   for (int i = 0; i<n; i++) {
        printf("Ingrese M[%d][%d]: ", i,j);
        scanf("%d", &M[i][j]);
   }
 }
...
for (int i = 0; i < n; i++) {
    free(M[i]);
}
free(M);
```

Cada llamada a `malloc` debe corresponder a una llamada a `free`.

DEPARTAMENTO DE
INGENIERÍA
INFORMÁTICA
UNIVERSIDAD DE SANTIAGO DE CHILE

Lectura de archivos
(guarda datos en **matriz**)

Dado un archivo con **n filas** y **m elementos columna** por cada fila. La primera fila guarda n y m. Guarde los datos en una matriz.

dataM.txt

```
10,3
7 3 3
3 5 4
6 6 7
4 4 3
21 20 21
9 12 15
55 56 57
45 30 30
33 2 1
20 4 12
```

```
NUM[][] leer_datos_fil(CHAR nom_archivo, NUM *n, NUM *m)
   FILE fp = OPENFILE(nom_archivo, "r")
   READFILE(fp,2,n, m) //lee desde fp 2 valores y los guarda en n y m
   NUM  matriz[*n][*m]
   FOR i=0 TO *n-1 DO
      FOR j=0 TO *m-1 DO
            READFILE(fp, 1, matriz[i][j])
   END-FOR
   CLOSEFILE(fp)
   RETURN  matriz
```

```
gcc leerFil_M.c  -o leerFil_M.exe
./leerFil_M.exe dataM.txt
```

```c
#include<stdio.h>
#include<stdlib.h>
int** leer_datos_Fil(char *nom_archivo, int *n, int *m) {
  FILE *fp = fopen(nom_archivo, "r");
  if (fp==NULL){
    printf("\n Archivo no existe\n");
    return NULL;
  }
  fscanf(fp, "%d,%d", n, m);
  int **matriz = malloc(n * sizeof(int *));
  for(int i=0;i<n;i++){
    matriz[i] = malloc(m * sizeof(int));
  }
  for (int i = 0; i < *n; i++)
    for (int i = 0; i < *n; i++)
        fscanf(fp, "%d", &matriz[i][j]);
  fclose(fp);
  return matriz;
}
int main(int argc,  char *argv[]){
    int n=0, m=0;
    if (argc != 2)
        return 1;
    int **M = leer_datos_Fil(argv[1], &n, &m);
    if ( M == NULL )
        return 1;
     ...
}
```

paso por referencia el **n y m** para para que pueda ver su valor actualizado aquí en el main

TERMINE: mostrar cada valor de la matriz con salto en cada fila y finalmente liberar memoria

Registros con struct y tipos de datos creados por usuario

Cree un registro para guardar las calificaciones de un estudiante de la asignatura EDA. **El registro debe tener** el **id** del estudiante, sus **notas de PEP** y de **controles.** Implemente un algoritmo que ingrese las notas de un id alumno dado, y guardelas en un archivo donde cada fila corresponde a un tipo de nota y las notas separadas por espacio.

```
CONST num_notas=3
RECORD notas_EDA:
    NUM id_alum,
    real peps[num_notas] ,        // ó real peps[]
    real contrs[num_notas],       // ó real contrs[]
END-RECORD
TYPE RECORD notas_EDA notasEDA;
ingresarNotas(NUM idEst)
    notasEDA E
    E.id_alum = idEst
    FOR i= 0 TO num_notas-1 DO
        READ(E.peps[i])
    FOR i= 0 TO num_notas-1 DO
        READ(E.contrs[i])
    FILE *fp = OPENFILE("salida.txt", "w")
    linea = ''
    FOR i= 0 TO num_notas-1 DO
        linea = linea + STR(E.peps[i]) + ' '
    WRITEFILE(fp, linea+'\n')
    linea = ''
    FOR i= 0 TO num_notas-1 DO
        linea = linea + STR(E.contrs[i]) + ' '
    WRITEFILE(fp, linea+'\n')
    CLOSEFILE(fp)
```

Si los arreglos se definieron con peps[] y contrs[], después de declarar E hacer:
E.peps = CREATE ARRAY(num_peps)
E.contrs = CREATE ARRAY(num_contrs)
Esto define que los arreglos tienen tamaño variable

```
#include<stdio.h>
#include<stdlib.h>
#define num_notas 3
struct notas_EDA{
   int id_alum;
   float peps[num_notas];
   float contrs[num_notas];
};
typedef struct notas_EDA notasEDA;
notasEDA ingresar_notas(int idalum){
  notasEDA E;
  E.id_alum = idalum;
  for (int i=0; i<num_notas; i++)
    scanf("%d", &E.peps[i]);
  ...
  FILE *fp = fopen("salida.txt", "w");
  for (int i=0; i<num_notas; i++)
    fprintf(fp, "%.2f ", E.peps[i]);
  fprintf(fp, "\n");
  for (int i=0; i<num_notas; i++)
    fprintf(fp, "%.2f ", E.contrs[i]);
  fprintf(fp, "\n");
  fclose(fp);
  return E;
```

Si los arreglos se definieron usando
float *peps
float *contrs
después de declarar E hacer:
E.peps = malloc(num_notas * sizeof(float));
E.contrs = malloc(num_notas * sizeof(float))
y agregue al struct num_notass

El archivo de escritura se usa similar a un printf y el open es con modo "w"

Su turno: Guia_lab_semana3

Descargue la guía de laboratorio guia_lab_semana3_EDAejec.pdf de la Uvirtual y desarrolle cada uno de los problemas presentados en esta guía.

Suba el desarrollo del laboratorio en el repositorio indicado por el docente.

Tarea para practicar lenguaje C

- **Construya los programas en C de los enunciados de las guías de la unidad**

Contenidos del siguiente laboratorio

- **Tipos de datos abstractos**
- **TDA Listas**